

SQL Server Inventory Template

1. Purpose

An outdated or incomplete database inventory is one of the most critical hidden risks in enterprise IT infrastructure. Without a centralized, reliable source of truth for every SQL Server instance, database administration teams face severe operational vulnerabilities during production outages, security audits, disaster recovery (DR) drills, and infrastructure migrations. The primary objective of the SQL Server Inventory Standard is to establish a unified, accurate, and continually updated repository of metadata across all SQL Server instances—whether hosted on-premises, across virtualized infrastructure, or within multi-cloud environments.

1.1. Key Operational Objectives

Accelerated Incident Management (MTTR): Drastically reduce Mean Time to Resolution during production outages by providing on-call DBAs immediate visibility into server specifications, application ownership, patch levels, and underlying storage layouts.

Continuous Audit & Compliance Readiness: Maintain an audit-ready baseline for compliance frameworks (SOC 2, ISO 27001, HIPAA, PCI-DSS) by tracking security configurations, authentication modes, patch baselines, and data retention policies.

Proactive Risk & End-of-Life (EOL) Management: Identify unsupported database engines, missing cumulative updates (CUs), unsafe configuration settings (e.g., clr enabled, default max server memory), and unhedged database risks before they trigger service degradation.

Disaster Recovery & Node Parity: Ensure failover nodes, AlwaysOn Availability Groups, and warm standby environments maintain matching build versions, collation settings, SQL Agent jobs, and security logins across secondary sites.

License & Capacity Optimization: Track CPU core counts, allocated memory, and edition distribution (Enterprise vs. Standard) to prevent licensing penalties and eliminate wasted compute resources.

2. Scope

This inventory framework applies to all SQL Server environments across the enterprise footprint. It provides a standardized data model to ensure complete operational visibility regardless of hosting architecture, deployment strategy, or environment tier.

2.1. Covered Hosting Architectures & Deployment Models

On-Premises Infrastructure: Bare-metal physical servers, VMware vSphere, and Microsoft Hyper-V virtual machines.

Infrastructure as a Service (IaaS): SQL Server running on cloud virtual machines (Azure Virtual Machines, AWS EC2, Google Cloud Compute).

Platform as a Service (PaaS): Azure SQL Managed Instance, Azure SQL Database, and AWS RDS for SQL Server (focusing on accessible engine DMVs and schema metadata).

Environment Tiers: Production (Prod), Disaster Recovery (DR), Staging (Stage), User Acceptance Testing (UAT), Quality Assurance (QA), and Development (Dev).

2.2. SQL Server Inventory Scope

1. Server & OS layer | Hostname, IP, OS Version, vCPUs, RAM, Disks
2. Engine & Instance | Build/CU, Edition, Service Accts, sys.config
3. Database & Storage | File Sizes, Recovery Model, Compat Level, TDE
4. HA/DR | AlwaysOn AGs, FCIs, Log Shipping Status
5. Security & Compliance | Sysadmins, Auth Mode, TLS Version, Audits
6. Automation & Jobs | SQL Agent Jobs, Operators, Linked Servers
7. Related Services | SSIS, SSRS, SSAS Configurations

2.3. Out-of-Scope Elements

Application Code: T-SQL stored procedure logic, user-defined functions, and application queries (maintained inside application source control repositories).

Non-SQL Server Engines: PostgreSQL, MySQL, Oracle, and NoSQL platforms (governed by engine-specific operational standards).

Ephemeral Test Containers: Containerized SQL Server instances (Docker/Kubernetes) created for automated CI/CD pipelines with lifespans under 24 hours.

3. Roles & Responsibilities

Keeping an inventory accurate requires teamwork. When ownership is unclear, data goes stale. The RACI matrix below defines responsibilities across departments:

- **R (Responsible):** The team doing the work to collect and maintain the data.
- **A (Accountable):** The ultimate decision-maker who ensures the inventory is correct.
- **C (Consulted):** Key stakeholders who provide necessary input or context.

- **I (Informed):** Teams who need read access to the inventory for daily operations.

3.1 RACI Matrix

Role / Team	DBA Team	Infrastructure	DevOps / App Team	InfoSec / Security	Management
Instance & Engine Metadata	A / R	C	I	I	I
Server Specs & OS Layer	C	A / R	I	I	I
App Ownership & Contacts	C	I	A / R	I	I
Security & Compliance Rules	R	I	I	A / C	I
EOL & Capacity Planning	A / R	C	C	I	I

3.2 Key Expectations

- **Database Administrators (DBAs):** Own the automation scripts, validate engine metadata monthly, and flag unbacked databases or EOL builds.
- **Infrastructure & Cloud Team:** Maintain accurate host-level details (vCPUs, host RAM, SAN storage, OS patch levels, network IPs).
- **DevOps & Application Teams:** Provide accurate application names, business criticality ratings, and emergency escalation contacts.
- **Information Security (InfoSec):** Set compliance targets (encryption standards, authentication policies) and review sysadmin access lists.
- **IT Management:** Resource the remediation of flagged risks (e.g., funding upgrades for EOL servers).

4. Inventory Categories

4.1 Server & OS Layer

Tracks the physical or virtual machine hosting the engine.

- **Key Fields:** Hostname, IP address, OS version, Domain/OU, CPU core count, Total RAM, Storage drive layout.
- **Why it matters:** Essential for host patching, hardware sizing, and OS lifecycle management.

4.2 Engine & Instance Layer

Captures server-wide database engine configuration settings.

- **Key Fields:** SQL Server version, Build/CU, Edition, Collation, Service Accounts, max server memory, cost threshold for parallelism, clr enabled.
- **Why it matters:** Prevents configuration drift and highlights missing security patches across servers.

4.3 Database & Storage Layer

Details of every individual database hosted on the instance.

- **Key Fields:** Database name, State (Online/Offline), Size (MB), Auto-growth settings, Recovery model, Compatibility level, TDE status.
- **Why it matters:** Helps forecast storage growth and prevent unexpected log file growth or performance regressions.

4.4 High Availability & DR Configuration

Maps redundancy and failover topologies.

- **Key Fields:** Topology type (AlwaysOn AG, FCI, Log Shipping), Node roles (Primary/Secondary), Availability mode (Sync/Async), Failover type.
- **Why it matters:** Ensures secondary nodes match the primary node's setup before a failover occurs.

4.5 Security & Compliance

Monitors access control and hardening settings.

- **Key Fields:** Authentication mode (Windows/Mixed), sysadmin role members, Service account privileges, TLS version, Audit spec status.
- **Why it matters:** Keeps the environment secure and audit-ready for compliance frameworks.

4.6 Feature Inventory

Catalogues extra SQL Server components running on the host.

- **Key Fields:** SSIS, SSRS, SSAS, PolyBase, Service Broker, Machine Learning Services, Replication.
- **Why it matters:** Ensures no hidden dependencies are missed during server migrations or upgrades.

4.7 Backup & Recovery Baseline

Records backup strategies and historical execution.

- **Key Fields:** Full/Diff/Log backup frequency, Backup paths, Encryption status, Last successful backup timestamps, RPO/RTO targets.
- **Why it matters:** Instantly exposes unbacked databases before a disaster strikes.

4.8 Job & Automation Schedule

Tracks automated background tasks and maintenance routines.

- **Key Fields:** SQL Agent jobs, Schedule cadence, Failed job alerts, Notification operators, Active Linked Servers.
- **Why it matters:** Ensures routine maintenance runs on schedule and documents cross-server data links.

4.9 Ownership & Application Contacts

Links database infrastructure to the business.

- **Key Fields:** Application name, Environment tier (Prod/QA/Dev), Business owner, technical owner, Emergency contact.
- **Why it matters:** Tells the on-call DBA exactly who to call when a system goes down.

4.10 Risk Flags & Health Baseline

Summarizes active vulnerabilities and operational concerns.

- **Key Fields:** End-of-Life (EOL) status, Missing backups, Over-allocated memory, Percent-growth settings, Untested DR nodes.
- **Why it matters:** Turns static inventory lists into a proactive action plan for the team.

5. Standard Inventory Fields

The target logical schema for storing collected inventory metadata across environments:

5.1 Host & OS Schema

Field Name	Data Type	Sample Value	Description
ServerName	VARCHAR(64)	SQLPROD01	Server netbios name
DomainName	VARCHAR(64)	CORP.LOCAL	Active Directory Domain
IPAddress	VARCHAR(45)	10.0.1.50	Primary binding IP address
OSVersion	VARCHAR(128)	Windows Server 2022	Operating system release
CPUCount	INT	16	Logical CPU cores
RAM_GB	DECIMAL(10,2)	64.00	Total physical RAM installed

5.2 Instance Schema

Field Name	Data Type	Sample Value	Description
InstanceName	VARCHAR(64)	MSSQLSERVER	Default or named instance
ProductVersion	VARCHAR(32)	16.0.4115.5	Full engine version build
ProductLevel	VARCHAR(32)	CU12	Cumulative Update level
Edition	VARCHAR(64)	Enterprise Edition (64-bit)	SQL Server SKU/Edition
Collation	VARCHAR(64)	SQL_Latin1_General_CP1_CI_AS	Instance default collation
MaxMemory_MB	INT	57344	Configured max server memory

5.3 Database Schema

Field Name	Data Type	Sample Value	Description
DatabaseName	VARCHAR(128)	ERP_Production	Name of database
StateDesc	VARCHAR(32)	ONLINE	Current operational state
RecoveryModel	VARCHAR(32)	FULL	FULL, BULK_LOGGED, SIMPLE
CompatLevel	INT	160	Database compatibility level
SizeMB	DECIMAL(10,2)	1048576.00	Total allocated size

Field Name	Data Type	Sample Value	Description
IsEncrypted	BIT	1	Transparent Data Encryption (TDE)

6. Inventory Collection Procedures

Inventory data should be collected automatically on a regular schedule rather than manually entered. This framework supports a hybrid automated collection process using native T-SQL DMVs for single-server execution and PowerShell for environment-wide discovery.

6.1 Collection Methods Summary

Method	Target Scope	Frequency	Output Storage /	Primary Tool
Local DMV Query	Single Instance	Daily / On-Demand	Central Admin DB / Table	Scripts/Inventory-TSQL.sql
Central Management Server (CMS)	Multi-Instance Group	Weekly	Aggregated SQL Table	SSMS + CMS Query Execution
PowerShell Automation	Enterprise Fleet	Daily (Scheduled Task)	CSV / Central SQL Database	Scripts/Inventory-PowerShell.ps1
Operational Health Audit	Security & Backups	Weekly	Audit Log / Risk Flags	Scripts/Inventory-Checks.sql

6.2 Recommended Execution Workflow

- Deployment:** Create a lightweight centralized database (e.g., DBA_Admin or DBA_Inventory) on a utility SQL Server instance.
- Scheduled Collection:** Run Inventory-PowerShell.ps1 via Windows Task Scheduler or a SQL Agent job off-peak daily.
- Local Pulls:** On non-domain or isolated SQL instances, execute Inventory-TSQL.sql locally and push results to the central inventory table over Linked Server or staging tables.
- Exception Handling:** Flag any target host that fails to respond within a 30-second timeout window for immediate network or service troubleshooting.

7. Validation Steps

An unvalidated inventory leads to blind spots. The following automated validation queries identify incomplete, stale, or rogue metadata records.

7.1 Validation Rules & Queries

- **Stale Record Check:** Flag any server whose inventory record has not been updated in the past **7 days**.
- **Orphaned Database Check:** Identify active databases missing an assigned Business Owner or Application Contact.
- **Service Account Parity Check:** Verify SQL Server Service accounts conform to current Managed Service Account (gMSA) or domain security standards.

---sql Inventory Validation: Description: Flag Stale Server Entries & Missing App Contacts.

```
SELECT ServerName, InstanceName, LastInventoryDate, DATEDIFF(DAY, LastInventoryDate, GETDATE())  
AS DaysOutOfDate, ApplicationName, BusinessOwner  
FROM DBA_Inventory.dbo.InstanceMetadata  
WHERE LastInventoryDate < DATEADD(DAY, -7, GETDATE()) OR  
ApplicationName IS NULL OR  
BusinessOwner IS NULL;
```

8. Review Cadence

To prevent operational drift, database administration teams must adhere to a strict review schedule across all tiers:

8.1 Operational Review Schedule

- **Weekly (Automated):** Execute Inventory-Checks.sql to catch missing backups, abnormal file growth, or newly created databases.
- **Monthly (DBA Team):** Audit Cumulative Update (CU) patch levels, verify non-production drift, and clean up stale inventory records.
- **Quarterly (Cross-Functional):** Review sysadmin access lists with InfoSec, evaluate disk space capacity trends with Infrastructure, and verify EOL build roadmaps.
- **Annually (Executive):** Review enterprise edition distributions, licensing core optimization, and multi-cloud strategies with IT Leadership.

9. Risk Assessment Checklist

Use this checklist during monthly reviews to identify high-risk conditions requiring immediate remediation:

Risk Trigger	Risk Severity	Threshold / Condition	Remediation Action
End-of-Life Engine	● High	SQL Server version extended support expired	Plan upgrade path to supported version
Unbacked Database	● High	No Full backup in 7 days (or Log in 4 hrs)	Verify backup agent and run immediate backup
Missing Max Memory	● Medium	max server memory left at default	Calculate and set threshold to prevent OS starvation
Auto-Percent Growth	● Medium	Database files set to grow by %	Reconfigure file auto-growth to explicit MB sizes
Untested DR Node	● Medium	AG secondary build version != Primary build	Apply missing CU to synchronize build levels

10. Best Practices

Establishing a consistent inventory standard requires strict automation and documentation hygiene across all database environments:

- **Automate Discovery & Collection:** Never rely on manual spreadsheets or one-off static lists. Schedule automated PowerShell scripts and T-SQL DMVs to poll metadata daily.
- **Enforce Naming Conventions:** Standardize hostnames, instance names, and environment tags (PROD, STAGE, UAT, DEV) across all servers to eliminate ambiguity.
- **Maintain Zero-Trust Access:** Store collected metadata in a centralized, read-only database with strict access control. Never record clear-text passwords, secrets, or sensitive connection strings.
- **Track Hardware & Configuration Changes:** Log changes to server vCPUs, memory allocations, or database file relocations immediately within the inventory repository.
- **Audit Document Hygiene:** Conduct periodic reviews to remove decommissioned servers and archive obsolete database records to maintain dataset integrity.

11. Metrics & KPIs

Track these key performance indicators to measure inventory health and overall operational risk reduction:

Metric Name	Target Objective	Measurement Method	Operational Value
Inventory Coverage Rate	100%	(Registered Instances / Total Infrastructure Fleet) × 100	Ensures zero shadow IT or unmonitored SQL servers.
Data Freshness Index	> 98%	% of records updated within the last 24 hours	Prevents reliance on stale hardware or schema data.
Unassigned Asset Rate	0%	Databases missing documented Business/Technical owners	Accelerates emergency escalation and MTTR during outages.
EOL Engine Exposure	0%	Production instances past Microsoft Extended Support	Highlights aging database engines requiring immediate upgrade.

12. Appendices

Reference operational assets and scripts located within the repository for automation:

- **T-SQL Inventory Collector:** 01-Foundation/SQL-Server-Inventory-Template/Scripts/Inventory-TSQL.sql
- **PowerShell Automation Engine:** 01-Foundation/SQL-Server-Inventory-Template/Scripts/Inventory-PowerShell.ps1
- **Risk & Health Audit Suite:** 01-Foundation/SQL-Server-Inventory-Template/Scripts/Inventory-Checks.sql

Zadig Negussie

SQL Server DBA Operations Library (SQLDBAOpsLib)

Date: August 31, 2026